



Universidad Nacional Experimental de Guayana.
Vicerrectorado Académico.
Coordinación General de Pregrado.
Proyecto de Carrera: Ingeniería en Informática.
Unidad Curricular: Ingeniería de Software II.

Liderazgo, Equipos y Cultura de Ingeniería.

Team Pixel & Code

“Estrategia, desarrollo y valor real”

Profesor:

Ms. Félix Márquez

Integrantes:

C.I.: V-28.530.717. Bravo Yvanna
C.I.: V-28.726.605. Guilarte Keibel
C.I.: V-19.505.397. Rivas Miguel
C.I.: V-24.848.424. Varguillas Génesis.

Ciudad Guayana, 23 de Septiembre del 2026.

Introducción

En la ingeniería de software suele pensarse que los problemas de entrega, calidad o innovación se resuelven con mejores herramientas, lenguajes o arquitecturas. Sin embargo, la experiencia de la industria y la investigación de las últimas décadas muestran que muchos de los obstáculos más persistentes son organizacionales y humanos. La forma en que se estructuran los equipos, la manera en que se comunican y el tipo de liderazgo que se ejerce sobre ellos determinan en gran medida qué software se construye, con qué rapidez y con qué calidad. Por eso el liderazgo, el diseño de equipos y la cultura de ingeniería son hoy un campo de estudio propio dentro de la disciplina.

El presente informe aborda estos temas a partir de dos casos que muestran dos dimensiones de un mismo problema. El primero corresponde a una empresa bancaria organizada en departamentos funcionales de interfaz, backend y base de datos, coordinados por un Gerente de Proyecto, cuyo sistema monolítico en capas sufre demoras de meses para entregar nuevas funcionalidades del área de Créditos. Para analizarlo recurrimos a la Ley de Conway (Conway, 1968), que explica por qué la arquitectura del sistema reproduce la estructura de comunicación de la organización. A partir de ese diagnóstico proponemos un rediseño basado en la Maniobra Inversa de Conway y en los tipos de equipo y modos de interacción de Team Topologies (Skelton y Pais, 2019), apoyado en el Diseño Guiado por el Dominio (Evans, 2003) y en el patrón de la higuera estranguladora (Fowler, 2004) para la migración gradual a microservicios. También redefinimos el rol del antiguo Gerente de Proyecto desde el liderazgo de servicio (Greenleaf, 1977).

El segundo caso se sitúa dentro de un equipo, donde los comentarios sarcásticos de un Arquitecto Principal en las revisiones de código han llevado a los desarrolladores junior a dejar de participar y han hecho caer la velocidad del equipo. Lo analizamos con el concepto de seguridad psicológica de Edmondson (1999), los hallazgos del proyecto Aristotle de Google y la tipología cultural de Westrum, relacionándolo con las métricas de entrega propuestas por Forsgren, Humble y Kim (2018). Luego planteamos una intervención en tres frentes: una conversación privada con el Arquitecto guiada por la Comunicación No Violenta, la reestructuración del proceso de revisión de código mediante automatización y Architecture Decision Records, y una dinámica de retrospectiva orientada a reconstruir la confianza.

El objetivo del trabajo es mostrar que la agilidad y el alto rendimiento no se logran adoptando ceremonias o tecnologías de forma aislada, sino alineando la estructura organizacional, la arquitectura del sistema y la cultura del equipo.

Pregunta 1 (Parte A): Análisis del problema mediante la Ley de Conway

La Ley de Conway como marco de análisis

En 1968, Melvin Conway formuló el principio que hoy lleva su nombre: "las organizaciones que diseñan sistemas están constreñidas a producir diseños que son copias de las estructuras de comunicación de dichas organizaciones" (Conway, 1968). La idea central es que la arquitectura de un sistema no es una decisión puramente técnica. Surge de cómo se comunican las personas que lo construyen. Si dos grupos no se hablan, sus componentes tampoco se integrarán bien. Si toda la comunicación pasa por un único canal, el sistema dependerá de ese mismo punto.

Aplicación al caso del banco

La empresa bancaria confirma la ley de forma casi literal. Tiene tres departamentos separados por especialidad técnica y un monolito dividido en tres capas. La correspondencia entre la organización y el sistema es directa:

Estructura organizacional	Reflejo en la arquitectura
Departamento de UI	Capa de presentación
Departamento de Backend Java	Capa de lógica de negocio
Departamento de Base de Datos	Capa de datos compartida por todo el sistema
Gerente de Proyecto como único canal de coordinación	Integración centralizada y fuertemente acoplada

La arquitectura monolítica no es un accidente ni una mala decisión técnica aislada. Es la consecuencia natural de la estructura funcional. Cada departamento optimiza su propia capa según sus criterios, pero ninguno es dueño de una capacidad de negocio completa como "Créditos". Esa funcionalidad existe repartida en tres capas, en tres departamentos y bajo tres jefaturas distintas.

Hay un agravante: la comunicación entre departamentos no es directa, sino que pasa por el Gerente de Proyecto. Según la clasificación del PMI (2021), en una estructura funcional el PM tiene poca autoridad, porque los recursos reportan a los gerentes de área. El PM termina actuando como intermediario sin poder real de decisión. Toda coordinación depende de él, lo que lo convierte en el cuello de botella de la organización.

El recorrido de una funcionalidad de "Créditos"

Cuando el negocio pide una nueva funcionalidad de créditos, el trabajo sigue una cadena secuencial. El Departamento de Base de Datos debe modificar el esquema. Después, el Departamento de Backend adapta la lógica. Por último, el Departamento de UI construye la interfaz. Cada paso implica una transferencia de contexto (handoff), y cada departamento tiene su propia cola de trabajo con prioridades que compiten entre sí.

El resultado es que el tiempo de entrega (lead time) no depende de cuánto se tarda en programar, sino de la suma de las esperas entre departamentos. Una funcionalidad que requiere pocos días de trabajo efectivo puede tardar meses en llegar a producción, porque pasa la mayor parte del tiempo esperando en colas. Por eso las demoras de meses del caso son una consecuencia estructural y no un problema de rendimiento individual.

¿Por qué la estructura garantiza el fracaso de la agilidad?

La agilidad se basa en entregar incrementos pequeños y frecuentes, obtener retroalimentación rápida y trabajar con equipos autónomos que se responsabilizan del resultado. La estructura funcional del banco hace imposible cada una de estas condiciones.

Autonomía. Ningún departamento puede entregar valor por sí solo. Cualquier cambio de negocio, por pequeño que sea, requiere la participación obligatoria de los tres departamentos. Un equipo no puede autoorganizarse si depende de otros dos para terminar su trabajo.

Entregas pequeñas y frecuentes. Como cada coordinación es costosa, la organización tiende a agrupar muchos cambios en lanzamientos grandes y espaciados. Esto contradice el principio ágil de lotes pequeños. Forsgren, Humble y Kim (2018) demostraron que el tiempo de entrega y la

frecuencia de despliegue son indicadores clave del alto rendimiento. En esta estructura, ambos se degradan necesariamente.

Retroalimentación rápida. Si el usuario detecta un problema, la corrección debe recorrer otra vez la cadena de tres departamentos. El ciclo de aprendizaje se vuelve tan lento que la organización no puede adaptarse al mercado, que es precisamente el propósito de la agilidad.

Responsabilidad compartida. Nadie responde por "Créditos" de extremo a extremo, así que cuando algo falla la responsabilidad se diluye. La UI culpa al backend, el backend culpa al esquema de datos y la base de datos culpa a requerimientos mal definidos. Este es el origen de los conflictos constantes del caso: los departamentos tienen objetivos locales distintos y ninguno tiene un objetivo común orientado al cliente.

Carga cognitiva. Skelton y Pais (2019) advierten que un equipo no debe cargar con más complejidad de la que puede manejar. Aquí ocurre lo contrario: la complejidad de la coordinación entre departamentos recae completa en el Gerente de Proyecto. Nadie en los equipos técnicos tiene una visión integral del producto.

Además, adoptar prácticas ágiles como sprints, reuniones diarias o tableros Kanban no resolvería el problema mientras la estructura siga igual. Cada departamento podría trabajar en sprints internamente, pero el flujo completo seguiría siendo secuencial y dependiente. Se tendría una agilidad aparente dentro de cada silo, sin agilidad real en la entrega de valor al cliente.

El círculo vicioso entre organización y arquitectura

La Ley de Conway también funciona en sentido inverso: una vez construido, el sistema refuerza la estructura que lo creó. Como el monolito está organizado en capas, el banco sigue necesitando especialistas por capa, lo que perpetúa los departamentos funcionales. Y como existen esos departamentos, cualquier cambio vuelve a producirse en capas. Se forma un círculo vicioso difícil de romper.

Esto tiene una consecuencia directa para el futuro. Si el banco intenta migrar a microservicios sin cambiar su organización, la Ley de Conway predice que obtendrá un monolito

distribuido: servicios separados físicamente pero acoplados en la práctica, que deben desplegarse juntos porque siguen reflejando las mismas fronteras departamentales. Tendría la complejidad de un sistema distribuido sin ninguno de sus beneficios.

Pregunta 1 (Parte B):

Rediseño organizacional con Team Topologies

El punto de partida del rediseño es la Maniobra Inversa de Conway, que consiste en diseñar primero la estructura de los equipos de acuerdo con la arquitectura que se quiere obtener, aprovechando que el sistema tiende a copiar la forma en que se comunica la organización (Skelton y Pais, 2019). Si el banco quiere llegar a una arquitectura de microservicios, los equipos deben organizarse alrededor de los dominios del negocio de Créditos, de modo que las fronteras entre equipos coincidan con las fronteras entre servicios, algo que la división actual por capas técnicas impide. Para identificar esos dominios usamos la noción de contexto delimitado del Diseño Guiado por el Dominio (Evans, 2003), que agrupa las partes del negocio que comparten un mismo lenguaje y un mismo modelo de datos.

Al revisar el ciclo de vida de un crédito se distinguen tres flujos de valor que el cliente percibe de principio a fin. El primero comprende la simulación y la solicitud del crédito, el segundo abarca la aprobación, el desembolso y la consulta del crédito activo, y el tercero cubre el pago de las cuotas, la gestión de la mora y la cobranza. Cada uno de estos flujos se asigna a un equipo alineado al flujo de valor (stream-aligned), que es el tipo de equipo central de Team Topologies y el que concentra la mayor parte de las personas. Estos equipos son multifuncionales y reúnen desarrolladores backend y de interfaz provenientes de los antiguos departamentos, un especialista en datos y una persona dedicada a la automatización de pruebas, de manera que pueden diseñar, construir, desplegar y mantener su parte del producto sin esperar a otros departamentos.

Existe, además, una parte del dominio que exige un conocimiento muy especializado, que es el cálculo del riesgo crediticio y de las tasas de interés. Este cálculo combina modelos estadísticos de puntuación de clientes con reglas regulatorias que cambian con frecuencia, y si cada equipo alineado al flujo tuviera que dominarlo, su carga cognitiva sería excesiva. Por esa razón proponemos un equipo de subsistema complicado (complicated-subsystem), el Equipo del

Motor de Riesgo y Tasas, que desarrolla este cálculo y lo ofrece a los demás equipos a través de una API, que es justamente el tipo de caso que la lectura de la unidad asocia con estos equipos cuando menciona los motores de cálculo.

Para que los equipos alineados al flujo puedan desplegar por su cuenta necesitan una infraestructura que no tengan que construir ni administrar ellos mismos. Esa es la función del Equipo de Plataforma Interna (platform), que ofrece como servicios de autoatención el pipeline de integración y despliegue continuo, los entornos de contenedores, las bases de datos gestionadas, la observabilidad y los servicios comunes de identidad y seguridad. Buena parte de las personas del antiguo Departamento de Base de Datos encuentra aquí un lugar natural, ya que pasa de atender solicitudes de cambio de otros departamentos a ofrecer la administración de bases de datos como un producto interno que cualquier equipo puede usar sin abrir un ticket.

Por último, la migración a microservicios y la adopción de prácticas como el despliegue continuo, las pruebas automatizadas o el diseño por dominios requieren habilidades que hoy no existen en el banco. Para cubrir esa brecha proponemos un equipo facilitador (enabling), el Equipo Facilitador de Arquitectura y DevOps, formado por pocas personas con experiencia en estos temas, que acompaña temporalmente a cada equipo alineado al flujo hasta que este puede trabajar por su cuenta y luego pasa al siguiente. Este equipo actúa como un consultor interno, por lo que no aprueba ni bloquea las decisiones de los demás, y su éxito se mide por lo rápido que los otros equipos dejan de necesitarlo.

Team Topologies define también la forma en que los equipos deben relacionarse, mediante tres modos de interacción que son la colaboración, el consumo de algo como servicio (X-as-a-Service) y la facilitación (Skelton y Pais, 2019). En la nueva estructura, los equipos alineados al flujo consumen como servicio tanto la plataforma como el motor de riesgo, el equipo facilitador trabaja con ellos en modo de facilitación y la colaboración directa se reserva para momentos puntuales, por ejemplo cuando dos equipos deben acordar el contrato de una API nueva. La Tabla 1 resume la conformación de cada equipo y la Figura 1 muestra cómo se relacionan entre sí.

Tabla 1. Conformación de los equipos propuestos.

Equipo	Tipo	Responsabilidad	Perfiles y origen de los integrantes
Simulación y Solicitud	Alineado al flujo	Simulación de créditos, solicitud en línea y carga de documentos del cliente.	Desarrolladores backend y de interfaz, un especialista en datos, una persona de automatización de pruebas y un Product Owner.
Aprobación y Desembolso	Alineado al flujo	Aprobación de la solicitud, desembolso, cronograma de cuotas y consulta del crédito activo.	El mismo perfil multifuncional del equipo anterior.
Pagos y Cobranza	Alineado al flujo	Pago de cuotas, gestión de la mora, refinanciación y cobranza.	El mismo perfil multifuncional del equipo anterior.
Motor de Riesgo y Tasas	Subsistema complicado	Modelos de puntuación de riesgo y cálculo de tasas según la regulación vigente.	Desarrolladores backend con formación matemática y analistas de riesgo del banco.
Plataforma Interna	Plataforma	Pipeline de CI/CD, contenedores, bases de datos gestionadas, observabilidad e identidad.	La mayoría del antiguo Departamento de Base de Datos y el personal de operaciones.
Facilitador de Arquitectura y DevOps	Facilitador	Acompañar la migración, el diseño por dominios y la adopción de prácticas DevOps.	Pocos integrantes, entre arquitectos y desarrolladores senior.

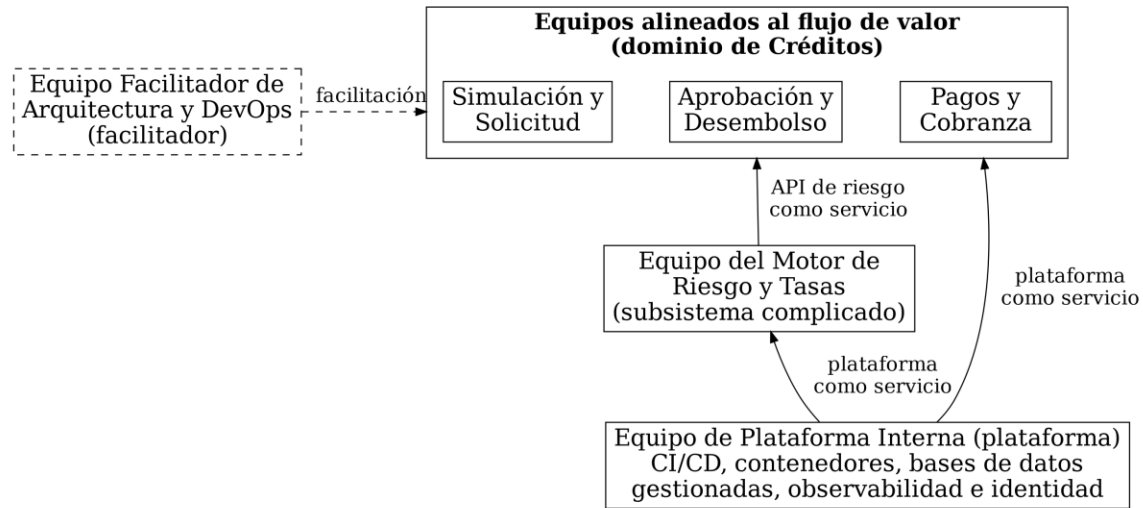


Figura 1. Estructura de equipos propuesta para el área de Créditos y sus modos de interacción.

Conformación de los equipos para la migración a microservicios

La estructura anterior está pensada para que la migración ocurra siguiendo las fronteras de los equipos. Cada equipo alineado al flujo será dueño de los microservicios de su dominio, de modo que el equipo de Simulación y Solicitud tendrá a su cargo los servicios de simulación y de solicitudes, el de Aprobación y Desembolso los servicios de desembolso y de cartera activa, y el de Pagos y Cobranza los servicios de pagos y de mora. Cada servicio tendrá su propia base de datos, administrada por el equipo dueño sobre la infraestructura que ofrece la plataforma, con lo cual desaparece el esquema compartido que hoy obliga a pasar cualquier cambio por el Departamento de Base de Datos. Como las conversaciones entre equipos quedan limitadas a los contratos de sus API, la misma Ley de Conway hace que las dependencias entre servicios queden igual de acotadas.

La migración tampoco tiene que hacerse de una sola vez. Proponemos aplicar el patrón de la higuera estranguladora (strangler fig), descrito por Fowler (2004), en el que las funcionalidades se van extrayendo del monolito una por una y se reemplazan por servicios nuevos, mientras el monolito sigue atendiendo todo lo que todavía no se ha migrado. Durante la transición, cada equipo alineado al flujo también es responsable de los módulos del monolito que pertenecen a su dominio, lo que evita que el sistema antiguo quede sin dueño. Un orden razonable es empezar por la simulación de créditos, que tiene poco riesgo porque no mueve

dinero, seguir con las solicitudes y dejar para el final el desembolso y los pagos, que son los procesos más sensibles, cuando los equipos ya tengan experiencia con el despliegue independiente.

Para mantener la carga cognitiva bajo control, cada equipo debe ser pequeño y estable, de entre cinco y nueve personas, que es el tamaño que Skelton y Pais (2019) consideran adecuado para que un grupo trabaje como una unidad con confianza mutua. Si más adelante un dominio crece demasiado, conviene dividirlo en un nuevo flujo con su propio equipo antes que seguir agregando personas al existente. Este modelo también cambia lo que se espera de cada integrante, porque un desarrollador que antes solo trabajaba en la interfaz ahora necesita entender la lógica y los datos de su dominio, lo que corresponde al perfil en forma de T que describe la lectura de la unidad.

El rol del antiguo Gerente de Proyecto

En la nueva estructura desaparece la función que justificaba al Gerente de Proyecto tradicional, que era coordinar a tres departamentos para que cada uno entregara su parte a tiempo, porque ahora cada equipo alineado al flujo puede llevar una funcionalidad de principio a fin sin esas transferencias. Aun así, la persona que ocupaba ese cargo conoce el negocio de Créditos, conoce a la gente de los tres antiguos departamentos y conoce las dependencias del sistema, y ese conocimiento sigue siendo valioso durante una transición tan larga como la migración a microservicios.

Proponemos que asuma el rol de líder de entrega del área de Créditos, entendido desde el liderazgo de servicio de Greenleaf (1977) que plantea la lectura de la unidad. En este rol deja de asignar tareas y de arbitrar conflictos entre áreas, y se dedica a remover los impedimentos que frenan a los equipos, a cuidar que los modos de interacción se respeten para que la colaboración entre dos equipos no se vuelva permanente y a hacer visible el flujo de trabajo mediante radiadores de información y métricas de entrega, como la frecuencia de despliegue y el tiempo que tarda un cambio en llegar a producción (Forsgren et al., 2018). También se convierte en el punto de contacto con las áreas del banco que siguen funcionando con reglas más formales, como cumplimiento, auditoría y riesgo operativo, y protege a los equipos de esa presión externa para que puedan concentrarse en entregar valor.

Este cambio es, en términos de Heifetz et al. (2009), un desafío adaptativo, porque exige modificar hábitos y formas de pensar más que aplicar una técnica conocida, y el antiguo gerente es quien mejor puede acompañar a las personas en ese aprendizaje, ya que él mismo tiene que recorrerlo. Si su perfil está más cerca del negocio que de la gestión de equipos, otra alternativa válida es que se convierta en Product Owner de uno de los flujos de Créditos, donde su conocimiento de las necesidades de los clientes y de las prioridades del banco le permitiría ordenar el backlog según el valor que aporta cada funcionalidad. En cualquiera de los dos casos, su influencia dependerá de cuánto ayude a los equipos a entregar, ya que en la nueva estructura no tendrá un plan que controlar ni departamentos que coordinar.

Con este rediseño, cada funcionalidad de Créditos deja de depender de tres departamentos y de un coordinador, porque un solo equipo puede llevarla desde la idea hasta producción apoyándose en la plataforma y en el motor de riesgo, y la organización del banco queda alineada con la arquitectura de microservicios que quiere construir.

Pregunta 2 (Parte A): Diagnóstico e Impacto a Largo Plazo de la Seguridad Psicológica

Diagnóstico de la situación

Amy Edmondson (1999) define la seguridad psicológica como la creencia compartida por los miembros de un equipo de que este es un entorno seguro para asumir riesgos interpersonales: preguntar, admitir un error, proponer una idea poco convencional o cuestionar una decisión, sin temor a ser humillado, ignorado o penalizado por ello. Edmondson identifica cuatro temores concretos que bloquean esta condición cuando no existe: el temor a ser visto como ignorante, como incompetente, como negativo o como disruptivo. El caso descrito constituye una violación directa de esta condición, y ocurre precisamente en el momento del ciclo de desarrollo donde la seguridad psicológica es más crítica: la revisión de código (Pull Request), instancia en la que el desarrollador expone de forma pública su razonamiento técnico ante sus pares.

Los comentarios del Arquitecto Principal ("¿En serio vas a usar este patrón? Es de principiantes") no constituyen retroalimentación técnica objetiva, sino un juicio sarcástico dirigido a la persona y no al código. Este tipo de comentario activa de manera directa el temor a

ser visto como incompetente, y lo hace además bajo una marcada asimetría de poder: quien emite el juicio es un desarrollador senior con alta autoridad técnica informal, mientras que quienes lo reciben son desarrolladores junior con escaso margen para replicar o defender su postura. El resultado observable, que los junior dejaron de proponer ideas en las retrospectivas, es exactamente lo que Edmondson describe como pérdida de "voz" (voice): el equipo aprende, de forma implícita, que el costo social de hablar es mayor que el beneficio de hacerlo, y opta por el silencio como estrategia de autoprotección.

Es importante señalar que el diagnóstico no debe reducirse a un problema de "personalidad difícil". Se trata de un problema estructural del proceso: no existen reglas objetivas de revisión de código, ni un canal que distinga la crítica técnica legítima del ataque personal, por lo que la autoridad técnica del Arquitecto se ejerce sin ningún contrapeso ni mecanismo de resguardo para los junior.

Impacto a largo plazo en el rendimiento del equipo

El proyecto Aristotle de Google (Duhigg, 2016) identificó la seguridad psicológica como el predictor número uno del alto rendimiento en equipos de software, por encima de la suma de talento individual. Su erosión en este caso genera consecuencias que se agravan con el tiempo:

Colapso del comportamiento de aprendizaje: Edmondson vincula la seguridad psicológica con el "learning behavior": hacer preguntas, pedir ayuda, discutir errores abiertamente y experimentar. Al desaparecer este comportamiento, los junior dejan de aprender de sus equivocaciones y de las del resto del equipo, y el conocimiento técnico se estanca en quienes ya lo poseen.

Retrospectivas vaciadas de contenido: Al perder su función de espacio honesto de mejora continua, la retrospectiva se convierte en un ritual sin sustancia: se cumple la ceremonia ágil, pero se pierde su propósito, y el equipo deja de detectar y corregir problemas de forma temprana.

Concentración del riesgo técnico en una sola persona: Al silenciarse la crítica de los junior, las decisiones del Arquitecto dejan de ser cuestionadas o verificadas por otros pares. Esto reduce, y no aumenta, la calidad técnica real del sistema: se pierde la diversidad de perspectivas que

permite detectar errores, y el equipo queda expuesto a un "bus factor" crítico concentrado en una única persona.

Caída sostenida de la velocidad de entrega: La caída de velocidad ya observada en el caso no es un hecho aislado, sino la consecuencia esperable: los junior, temerosos de ser señalados, evitan tomar iniciativa, escalan decisiones simples hacia el Arquitecto y sobre-consultan antes de actuar, generando cuellos de botella y dependencias que el propio modelo de equipos multifuncionales busca eliminar.

Riesgo de rotación de talento: Los desarrolladores junior, al ser el eslabón con menor poder relativo y mayor exposición al maltrato, son también los que tienen mayor probabilidad de abandonar el equipo o la organización, lo que implica pérdida de conocimiento y costos de reemplazo.

Degradación hacia una cultura organizacional patológica y caída de las métricas DORA: Westrum (como se citó en Forsgren et al., 2018) clasifica las culturas organizacionales en patológica, burocrática y generativa, según cómo circula la información: en la primera, la información se oculta, se castiga al mensajero y el fracaso se gestiona buscando culpables. Un equipo donde los junior temen señalar un problema deja de reportar riesgos a tiempo, lo que en la práctica reduce la frecuencia de despliegue, incrementa el lead time y eleva la tasa de fallos en producción, es decir, degrada directamente las métricas DORA que la propia organización utiliza como radiador de información. La seguridad psicológica deja así de ser un asunto exclusivamente de clima laboral para convertirse en una variable de ingeniería con efecto medible sobre la entrega de software.

Efecto sobre la arquitectura del sistema (Ley de Conway): Al concentrarse todas las decisiones técnicas en el Arquitecto y silenciarse al resto del equipo, la estructura de comunicación real del equipo deja de ser colaborativa para convertirse, de facto, en una estructura de un único nodo de decisión. Por la Ley de Conway, esta topología de comunicación tiende a reproducirse en el software: el sistema queda acoplado al criterio de una sola persona, en lugar de reflejar la arquitectura distribuida y desacoplada que buscan los stream-aligned teams

analizados en la Pregunta 1, lo que compromete a mediano plazo la propia evolución técnica del producto.

En síntesis, de no intervenir, el equipo se encamina hacia una cultura de miedo que sustituye a la cultura de aprendizaje: la innovación se detiene, la deuda técnica se acumula de forma silenciosa porque nadie se atreve a señalarla, y el desgaste (burnout) de los desarrolladores junior se vuelve cada vez más probable.

Este diagnóstico deja planteada la necesidad de una intervención estructurada que transforme la autoridad técnica del Arquitecto en liderazgo de servicio y mentoría, y que dote al proceso de revisión de código de reglas objetivas que separen la crítica técnica del juicio personal. El diseño concreto de dicha intervención, apoyado en la Comunicación No Violenta, la reestructuración del flujo de revisión y una dinámica de retrospectiva orientada a reconstruir la confianza, se desarrolla en la Parte B del presente trabajo.

Pregunta 2 (Parte B)

Estrategias de solución

1.- Estrategia de Intervención en Privado

Objetivo: Aplicar Liderazgo de Servicio / Adaptativo para transformar el rol del Arquitecto hacia la mentoría e influencia técnica mediante la Comunicación No Violenta (CNV) (Rosenberg, 2015).

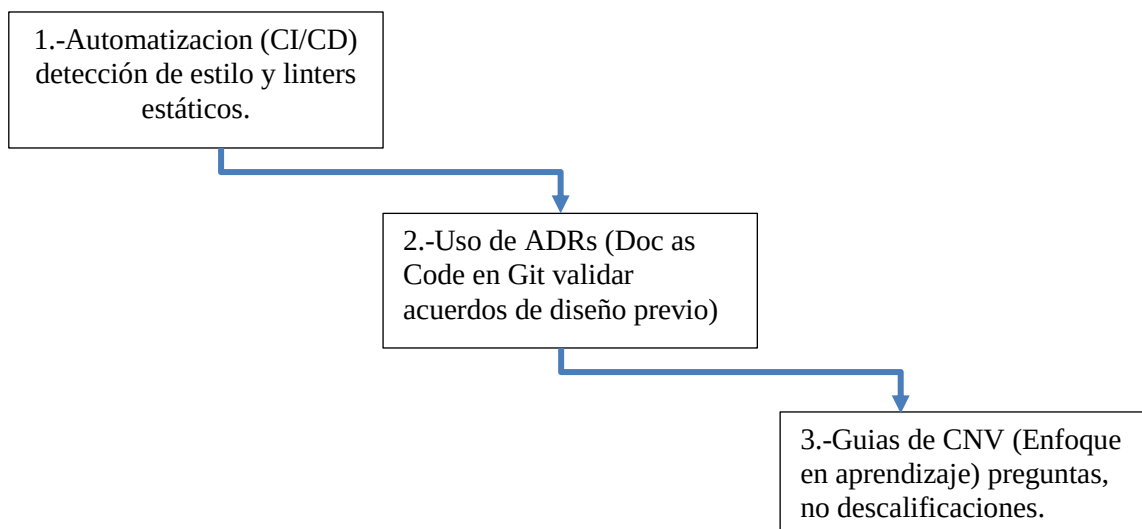


Fig. Paradigmas de la CNV.
Fuente: Management 3.0

Fase CNV	Guión de Intervención	Propósito
1. Hechos (Sin juzgar)	"He notado en los últimos Pull Requests comentarios como 'Es de principiantes'."	Evita una actitud defensiva al señalar datos objetivos y no juicios de valor.
2. Impacto / Diagnóstico	"Esto está destruyendo la Seguridad Psicológica ; los juniors temen participar y la velocidad cayó."	Visibiliza el impacto negativo en la autonomía y la cohesión del equipo.
3. Necesidad Técnica	"Para desarrollar perfiles T-shaped , tu rol debe ser de guía y persuasión, no de imposición."	Alinea el expertise del Arquitecto con el Liderazgo Técnico basado en la enseñanza.
4. Petición / Acción	"Canalicemos los cambios complejos vía ADRs y prototipos en lugar de críticas punitivas."	Fomenta el uso de Documentation as Code para objetivar el diseño.

2.- Reestructuración de la Revisión de Código

Objetivo: Convertir el conflicto interpersonal destructivo en un conflicto técnico saludable (Thomas y Kilmann, 1974).



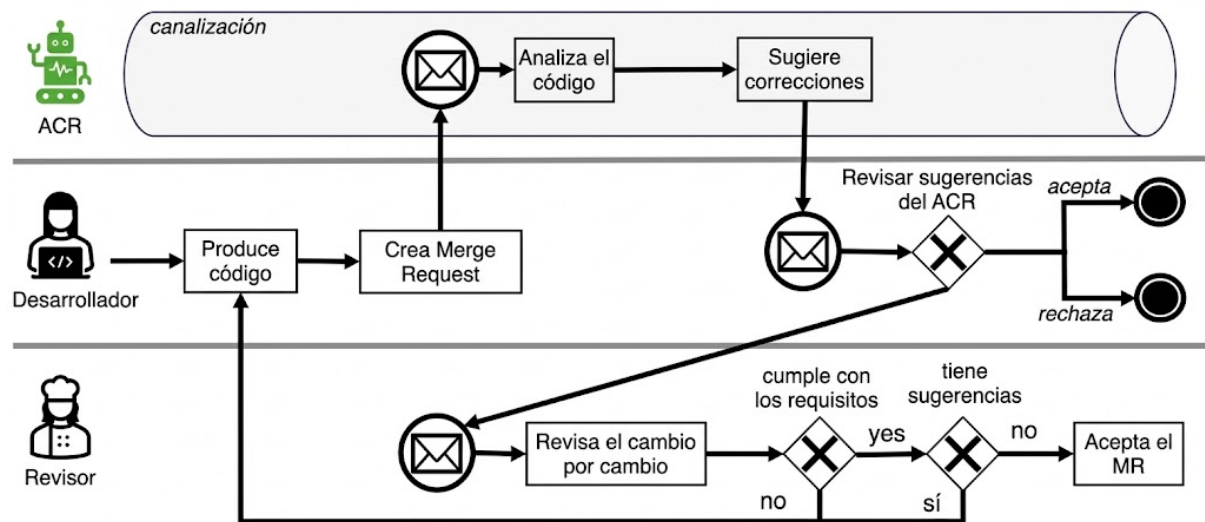


Fig 2. Flujo objetivo de revisión de código.

Fuente: BL Research

Pilar del Nuevo Proceso	Acción Concreta	Herramienta / Práctica
Automatización	Delegar formato y estilo a la tubería de integración.	Linters y pruebas automáticas en CI/CD.
Estandarización	Evaluar el código contra decisiones acordadas, no gustos.	Architecture Decision Records (ADRs) en Git.
Feedback Orientado a Aprendizaje	Cambiar críticas destructivas por preguntas que generen reflexión.	Sustituir <i>"Es de principiantes"</i> por <i>"¿Qué impacto tendría esto en latencia?"</i> .

3.- Dinámica de Retrospectiva

Nombre de la dinámica: *Lienzo de Seguridad Psicológica y Matriz de Acuerdos*

Objetivo: Reconstruir la confianza, motivar mediante la autonomía y hacer visibles los compromisos.

Etapa	Actividad	Detalle Técnico
1. Diagnóstico Anónimo	Extracción <i>Pull</i> de información sobre barreras y temores del equipo.	Uso de herramientas asíncronas para garantizar un entorno seguro sin exposición.
2. Co-creación de Alianza	Debate abierto para redefinir las reglas de interacción en el equipo.	Transición de conflictos interpersonales hacia compromisos de respeto y maestría.
3. Cierre y Visibilidad	Publicación del <i>Team Working Agreement</i> a la vista de todos.	Implementación como un Radiador de Información digital para consulta continua.

En conclusión, transformar un entorno de revisión de código punitivo en un espacio de alto rendimiento exige reemplazar la imposición jerárquica por un Liderazgo de Servicio y Adaptativo, donde la brillantez técnica se encauce hacia la mentoría mediante la Comunicación No Violenta y la formalización de acuerdos con *Architecture Decision Records* (ADR). Al objetivar el *feedback* mediante reglas explícitas y herramientas automatizadas, y al institucionalizar dinámicas de retrospectiva respaldadas por radiadores de información, se restituye la Seguridad Psicológica necesaria para que los desarrolladores recuperen la autonomía, la motivación intrínseca y la capacidad de innovar sin temor a ser desacreditados.

Conclusión

Los dos casos analizados conducen a una misma idea: el rendimiento de un equipo de software depende tanto de cómo está organizado y de cómo se relacionan sus integrantes como de su capacidad técnica. En el banco, la lentitud en la entrega no se explicaba por falta de talento, sino por una estructura funcional que obligaba a cada funcionalidad de Créditos a recorrer tres departamentos y a depender de un coordinador sin autoridad real. La Ley de Conway permitió ver que el monolito en capas era el reflejo de esa organización, y que cualquier intento de migrar a microservicios sin cambiar la estructura terminaría en un monolito distribuido. Por eso la solución propuesta empieza por los equipos y no por el código: equipos alineados al flujo de valor, apoyados por una plataforma interna, un equipo de subsistema complicado para el motor de riesgo y un equipo facilitador, con modos de interacción definidos y una migración gradual que sigue las fronteras de cada dominio.

En el segundo caso, el problema no estaba en la estructura formal, sino en la cultura. Un solo comportamiento, la crítica sarcástica dirigida a la persona y no al código, bastó para silenciar a los desarrolladores junior, vaciar las retrospectivas y concentrar todas las decisiones técnicas en una sola persona. El análisis mostró que la pérdida de seguridad psicológica no es solo un asunto de clima laboral, sino una variable de ingeniería con efectos medibles sobre la velocidad, la calidad, la retención del talento e incluso la arquitectura del sistema, ya que un equipo con un único nodo de decisión tiende también a producir software acoplado a ese nodo. La intervención propuesta combina una conversación basada en la Comunicación No Violenta, reglas objetivas de revisión apoyadas en automatización y ADRs, y una retrospectiva que convierte los acuerdos del equipo en un compromiso visible.

Ambos casos comparten además una lección sobre el liderazgo. Tanto el antiguo Gerente de Proyecto como el Arquitecto Principal tienen conocimiento valioso para la organización, pero ese conocimiento solo genera valor cuando se pone al servicio de los equipos en lugar de usarse para controlar o imponer. El paso de un liderazgo basado en la autoridad a uno basado en el servicio, la mentoría y la influencia técnica es, como señalan Heifetz et al. (2009), un desafío adaptativo que exige cambiar hábitos y formas de pensar.

En definitiva, este trabajo muestra que estructura, arquitectura y cultura no pueden tratarse por separado. Una organización que quiera entregar valor de forma rápida y sostenida necesita equipos autónomos con fronteras claras, un sistema cuya arquitectura acompañe esas fronteras y un entorno donde las personas puedan preguntar, equivocarse y proponer sin miedo. Solo cuando estas tres condiciones se cumplen a la vez, la agilidad deja de ser un conjunto de ceremonias y se convierte en una forma real de trabajar.

Referencias Bibliográficas

Conway, M. E. (1968). How do committees invent? *Datamation*, 14(4), 28–31.

Edmondson, A. C. (1999). Psychological safety and learning behavior in work teams. *Administrative Science Quarterly*, 44(2), 350–383.

Evans, E. (2003). *Domain-driven design: Tackling complexity in the heart of software*. Addison-Wesley.

Forsgren, N., Humble, J. y Kim, G. (2018). *Accelerate: The science of lean software and DevOps*. IT Revolution Press.

Greenleaf, R. K. (1977). *Servant leadership: A journey into the nature of legitimate power and greatness*. Paulist Press.

Heifetz, R., Grashow, A. y Linsky, M. (2009). *The practice of adaptive leadership*. Harvard Business Press.

Project Management Institute. (2021). *A guide to the project management body of knowledge (PMBOK guide)* (7.^a ed.). Project Management Institute.

Rosenberg, M. B. (2015). *Nonviolent communication: A language of life* (3.^a ed.). PuddleDancer Press.

Skelton, M. y Pais, M. (2019). *Team topologies: Organizing business and technology teams for fast flow*. IT Revolution Press.

Thomas, K. W. y Kilmann, R. H. (1974). *Thomas-Kilman conflict mode instrument*. Xicom.